

TECHNICAL DOCUMENTATION

Cybersecurity Analytics Platform

Project : The Network Bouncer: Detecting Suspicious Port Scanning in Data Center Traffic

Date of submission -: 22 June, 2026

Event : Dell Futureminds AI Hackathon

Document Type : Technical Document

PROJECT TEAM

Raj Rathaur : Team Lead and Backend Development

Anjali Kumari : Detection Logic and ML

Vanshika Dixit : Feature Engineering

Sonam Sharma : Dataset Analysis/Research and Testing

Prashant Sharma : Dashboard Development

TABLE OF CONTENTS

- 1. Project Overview**
- 2. System Architecture**
 - 2.1 Architecture Overview
 - 2.2 Pipeline Flow
 - 2.3 Module Breakdown
- 3. Data Layer**
 - 3.1 Dataset Description
 - 3.2 Feature Catalogue
 - 3.3 Raw Log Support
- 4. Detection Engine**
 - 4.1 Rule-Based Detection
 - 4.2 Risk Scoring & Classification
 - 4.3 ML-Based Detection
 - 4.4 Hybrid Decision Logic
- 5. Machine Learning Model**
 - 5.1 Model Architecture
 - 5.2 Training & Evaluation
 - 5.3 Model Artifact
- 6. Reporting Module**
 - 6.1 CSV Report Schema
 - 6.2 CLI Interface
- 7. Dashboard-**
 - 7.1 Streamlit UI
 - 7.2 Dashboard Components
- 8. Installation & Setup**
- 9. Testing**
- 10. Security Considerations**
- 11. Limitations & Future Work**
- 12. References & Dependencies**

1 PROJECT OVERVIEW

Network Bouncer is an open-source cybersecurity analytics platform designed to detect suspicious scan-like behavior in network traffic. Developed for the Dell Hackathon 2024, the system ingests UNSW-NB15-style CSV traffic data, computes behavioral indicators, and applies a hybrid rule-based and machine-learning detection strategy to surface high-risk hosts and traffic patterns.

Objectives

- Identify hosts or records contacting many destinations (port/destination scan behavior).
- Detect abnormal connection volume, port diversity, and repeated source-destination patterns.
- Provide explainable, rule-based detections alongside ML-powered attack predictions.
- Deliver findings through a structured CSV report and an interactive Streamlit dashboard.

Key Capabilities

Capability	Description
Behavioral Feature Extraction	Derives destination diversity, port diversity, connection volume, and repeat scores.
Rule-Based Detection	Explainable threshold-driven scoring with configurable CLI parameters.
ML-Based Detection	Stacking classifier (XGBoost + LightGBM + CatBoost + LR) with probability threshold
Hybrid Decision	Combines rule score and ML probability into a final classification.
CSV Report Output	Structured suspicious-activity report with severity, classification, and reason column
Streamlit Dashboard	Interactive upload-and-filter UI with KPI cards and optional ML insight widgets.
Full Test Suite	pytest coverage for parser, features, detection, ML scoring, reporting, and dashboa

2 SYSTEM ARCHITECTURE

2.1 Architecture Overview

The system follows a linear pipeline architecture with an optional ML branch. All components are implemented as independent Python modules under the **src/** package, ensuring clean separation of concerns and independent testability.

Architecture Style: Modular Pipeline | Language: Python 3.x | Interface: CLI (main.py) + Web UI (Streamlit) | Storage: File-based (CSV in / CSV out) | ML Runtime: scikit-learn / XGBoost / LightGBM / CatBoost

2.2 Pipeline Flow

Stage	Module / File	Input	Output
1. CSV Ingestion	src/parser/csv_parser.py	Raw CSV file path	Validated DataFrame
2. Feature Extraction	src/features/host_features.py	Validated DataFrame	Feature-enriched DataFrame
3a. Rule Detection	src/detection/rules.py	Feature DataFrame	Risk scores + reason strings
3b. ML Scoring*	src/detection/ml_model.py	Feature DataFrame + .pkl model	ml_prediction + probability
4. Risk Classification	src/detection/rules.py	Scores (+ optional ML output)	Classification + severity
5. Report Writing	src/reporting/report_writer.py	Classified DataFrame	CSV report file
6. Dashboard (optional)	src/dashboard/dashboard.py	Uploaded CSV report	Interactive web UI

* Stage 3b is activated with the `--use-ml` CLI flag.

2.3 Module Breakdown

Module	Path	Responsibility
CSV Parser	src/parser/csv_parser.py	Load, validate, and type-cast input CSV against schema.
Host Features	src/features/host_features.py	Compute <code>ct_dst_ltm</code> , <code>ct_src_dport_ltm</code> , <code>ct_dst_sport_ltm</code> , <code>ct_dst_src_ip</code>
Rules	src/detection/rules.py	Score each record; map score to classification + severity + reason.
ML Model	src/detection/ml_model.py	Load <code>.pkl</code> , encode categoricals, run <code>predict_proba</code> , apply threshold.
Report Writer	src/reporting/report_writer.py	Assemble final DataFrame, write to CSV, print console summary.
Dashboard	src/dashboard/dashboard.py	Streamlit entrypoint; file upload, filter widgets, KPI cards.
Report Processing	src/dashboard/report_processing.py	Filter by severity, compute summary statistics, detect ML columns.

3 DATA LAYER

3.1 Dataset Description

Network Bouncer uses cleaned UNSW-NB15 dataset files. The UNSW-NB15 dataset was created by the Australian Centre for Cyber Security (ACCS) and contains real modern attack traffic synthesised with normal network traffic. The cleaned versions included in this project contain derived behavioral features and ground-truth labels but do not expose raw IP addresses.

File	Path	Purpose
Training Set	data/cleaned/UNSW_NB15_training-set(in).csv	Model training and categorical encoding reference.
Testing Set	data/cleaned/UNSW_NB15_testing-set(in).csv	Default input for CLI demo and evaluation.
Schema Map	data/schema/UNSW-NB15_features(in).csv	Column name, type, and description reference.
Processed	data/processed/ (gitignored)	Generated suspicious-activity CSV reports.

NOTE: Because the cleaned CSVs do not include real **srcip**/**dstip**/**dsport** columns, Network Bouncer reports rows as record-**<id>**. When a raw log file with real IP columns is provided via CLI, the same pipeline reports actual source IPs.

3.2 Feature Catalogue

The following behavioral columns are central to detection logic:

Column	Full Name	Detection Relevance
ct_dst_ltm	Destination Count (Last Time)	Destination diversity — high values indicate scanning.
ct_src_dport_ltm	Src→Dest Port Count (Last Time)	Port diversity — indicates port sweep behavior.
ct_dst_sport_ltm	Dest Source Port Count (Last Time)	Source-port variation against destinations.
ct_dst_src_ltm	Dest–Source Pair Count (Last Time)	Repeated source-destination relationship — persistence.
ct_state_ttl	State/TTL Combination Count	Unusual protocol state / TTL combinations.
attack_cat	Attack Category Label	Ground-truth label — included in report output.
label	Binary Label (0=Normal, 1=Attack)	Ground-truth binary label — included in report output.
dur	Connection Duration	Input to ML model (42-feature vector).
proto	Protocol (Categorical)	Encoded for ML scoring.
service	Service (Categorical)	Encoded for ML scoring.
state	Connection State (Categorical)	Encoded for ML scoring.

3.3 Raw Log Support

When a raw log file is provided containing **srcip**, **dstip**, and **dsport** columns, Network Bouncer aggregates records by source IP and computes equivalent behavioral features. Raw logs can use the rule-based detection layer but *cannot* use the ML model, which requires all 42 UNSW-NB15 features.

4 DETECTION ENGINE

4.1 Rule-Based Detection

The rule-based detector (**src/detection/rules.py**) is the primary, always-active detection layer. It is explainable, deterministic, and operates without any trained model. Each record is scored on five behavioral dimensions:

Dimension	Feature Used	Default Threshold	Points Awarded
Destination Diversity	ct_dst_ltm	50 unique dsts	Scaled 0–40 pts
Port Diversity	ct_src_dport_ltm	30 unique ports	Scaled 0–30 pts
Connection Volume	ct_dst_src_ltm	100 connections	Scaled 0–20 pts
Repeat Src-Dst	ct_dst_src_ltm	High repeat count	Up to 5 pts
State / TTL Anomaly	ct_state_ttl	Unusual combos	Up to 5 pts

4.2 Risk Scoring & Classification

Scores from all dimensions are summed to a composite risk score (0–100):

Score Range	Classification	Severity	Meaning
0	Normal	Low	No suspicious behavioral indicators detected.
1 – 34	Watch	Low	Minor anomalies; monitor for persistence.
35 – 69	Suspicious	Medium	Significant indicators; investigate further.
70 – 100	High Risk	High	Strong scan-like or attack behavior detected.

4.3 ML-Based Detection

When **--use-ml** is passed, the ML scoring module loads the stacking classifier from **models/network_bouncer_model.pkl** and produces an attack probability for each record. The module applies ordinal encoding to categorical columns (*proto*, *service*, *state*) using the training CSV as a reference, ensuring consistent encoding between training and inference.

ML Threshold default: 0.65 (matches training notebook evaluation threshold). Override: **--ml-threshold <float>** | Categorical encoding: derived from UNSW_NB15_training-set(in).csv at runtime.

4.4 Hybrid Decision Logic

When both rule and ML scores are available, a **hybrid_decision** column is added to the report using the following logic:

- If rule classification = High Risk OR ml_prediction = 1 → hybrid_decision = High Risk.
- If rule classification = Suspicious AND ml_prediction = 0 → hybrid_decision = Suspicious.
- If rule classification = Watch AND ml_prediction = 1 → hybrid_decision = Suspicious.
- Otherwise → hybrid_decision follows the rule classification.

5 MACHINE LEARNING MODEL

5.1 Model Architecture

The included model is a **Stacking Classifier** — an ensemble meta-learner that combines predictions from multiple base estimators using a logistic regression meta-model.

Component	Algorithm	Role
Base Estimator 1	XGBoost (XGBClassifier)	Gradient-boosted trees — high recall on attack patterns.
Base Estimator 2	LightGBM (LGBMClassifier)	Fast gradient boosting — handles large feature sets.
Base Estimator 3	CatBoost (CatBoostClassifier)	Native categorical handling, ordered boosting.
Meta-Learner	Logistic Regression	Combines base estimator probabilities into final prediction.
Imbalance Handling	imbalanced-learn (SMOTE)	Oversampling minority (attack) class during training.

5.2 Training & Evaluation

Model training and evaluation are documented in `notebooks/Detecting_suspicious_port_scanning.ipynb`. The training pipeline includes:

- Feature selection from all 42 UNSW-NB15 behavioral and statistical columns.
- Ordinal encoding of categorical features (proto, service, state) using the training set.
- SMOTE oversampling to address class imbalance between normal and attack records.
- Cross-validated hyperparameter tuning for each base estimator.
- Final evaluation at probability threshold 0.65, producing precision / recall / F1 metrics.

5.3 Model Artifact

The serialised model is stored as a joblib-pickled sklearn pipeline:

COMMAND / CODE

```
models/network_bouncer_model.pkl

Serialization: joblib (sklearn-compatible)
Input:        42 UNSW-NB15 feature columns
Output:       Binary label (0/1) + attack probability
Encoding ref: data/cleaned/UNSW_NB15_training-set(in).csv
Threshold:    0.65 (configurable via --ml-threshold)
```

6 REPORTING MODULE

6.1 CSV Report Schema

The output report is written to **data/processed/suspicious_activity_report.csv** (default path, overridable with `--output`). Column definitions:

Column	Type	Always Present	Description
srcip	string	Yes	Source IP address, or record- <code><id></code> for cleaned CSV inputs.
classification	string	Yes	Normal / Watch / Suspicious / High Risk.
severity	string	Yes	Low / Medium / High.
risk_score	integer	Yes	Composite rule score, 0–100.
connection_count	integer	Yes	Total connections observed for this source.
unique_dstip_count	integer	Yes	Number of unique destination IPs contacted.
unique_dsport_count	integer	Yes	Number of unique destination ports contacted.
reason	string	Yes	Human-readable explanation of why the record was flagged.
attack_cat	string	Conditional	Ground-truth attack category label (if present in input).
label	integer	Conditional	Ground-truth binary label (if present in input).
ml_prediction	integer	--use-ml only	Binary ML prediction (0=Normal, 1=Attack).
ml_prediction_label	string	--use-ml only	Human-readable ML prediction label.
ml_attack_probability	float	--use-ml only	Raw attack probability from stacking classifier.
hybrid_decision	string	--use-ml only	Combined rule + ML final classification.

6.2 CLI Interface

All detection parameters are exposed as CLI arguments to **main.py**:

Argument	Default	Description
<input_csv>	testing CSV	Positional. Path to the input CSV file.
--output <path>	data/processed/ suspicious_activity_report.csv	Output Path.
--dst-threshold <n>	50	Min unique destination IPs to score destination-diversity risk.
--port-threshold <n>	30	Min unique destination ports to score port-diversity risk.
--connection-threshold	100	Min connections to score high connection-volume risk.
--include-normal	False	Include Normal-classified rows in the CSV report.
--use-ml	False	Enable ML stacking classifier scoring.
--model-path <path>	models/ network_bouncer_model.pkl	Explicit model file path.
--encoding-reference	training CSV	CSV used to derive categorical encodings for ML.
--ml-threshold <f>	0.65	Probability threshold for positive (attack) ML prediction.

Example hybrid detection run:

COMMAND / CODE

```
python main.py "data/cleaned/UNSW_NB15_testing-set(in).csv" \
  --output data/processed/report.csv \
  --use-ml \
  --model-path models/network_bouncer_model.pkl \
  --encoding-reference "data/cleaned/UNSW_NB15_training-set(in).csv" \
  --ml-threshold 0.65 \
  --dst-threshold 50 --port-threshold 30 --connection-threshold 100
```

7 DASHBOARD

7.1 Streamlit UI

The Streamlit dashboard (**src/dashboard/dashboard.py**) provides an interactive post-analysis interface. Analysts upload a generated CSV report and explore findings through filter controls and KPI cards.

COMMAND / CODE

```
# Launch dashboard
streamlit run src/dashboard/dashboard.py

# Then open: http://localhost:8501
# Upload: data/processed/suspicious activity report.csv
```

7.2 Dashboard Components

Component	Module	Behaviour
File Upload	dashboard.py	Accepts any CSV report generated by main.py.
Severity Filter	report_processing.py	Multi-select: HIGH / MEDIUM / LOW. Empty selection hides all rows.
KPI Cards	dashboard.py	Total Alerts / High Risk / Suspicious / Watch counts.
Data Table	dashboard.py	Filtered record view with all report columns.
ML Insight Widgets	dashboard.py	Appear conditionally only when ML columns are present in the report.
Summary Statistics	report_processing.py	Aggregated counts and average risk score per severity band.

8 INSTALLATION & SETUP

Prerequisites

- Python 3.9 or later
- Windows (PowerShell) or Linux/macOS (bash)
- Git (to clone the repository)

Installation Steps

COMMAND / CODE

```
# 1. Navigate to project directory
cd "C:\Users\<User>\Desktop\Dell Hackathon\dell-network-bouncer"

# 2. Create virtual environment
python -m venv .venv

# 3. Activate (Windows PowerShell)
.venv\Scripts\Activate.ps1

# 4. Upgrade pip and install dependencies
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Pinned Dependencies

Package	Version	Purpose
pandas	2.3.3	DataFrame manipulation and CSV I/O
numpy	2.2.6	Numerical operations
scikit-learn	1.6.1	ML pipeline, stacking classifier, encoding
xgboost	3.2.0	Gradient-boosted tree base estimator
lightgbm	4.6.0	Fast gradient-boosting base estimator
catboost	1.2.10	Categorical-native gradient-boosting estimator
imbalanced-learn	0.14.2	SMOTE oversampling for class imbalance
joblib	1.5.3	Model serialization / deserialization
streamlit	1.58.0	Dashboard web application
matplotlib	3.10.9	Plotting support (optional charts)
pytest	9.1.1	Test runner

9 TESTING

Network Bouncer includes a comprehensive pytest test suite under the **tests/** directory. Tests cover every layer of the pipeline and are designed to run without network access or external services.

Test File	Module Tested	What Is Covered
test_parser.py	src/parser/csv_parser.py	CSV loading, schema validation, dtype casting, missing-column err
test_features.py	src/features/host_features.py	Feature aggregation from raw srcip/dstip logs and cleaned CSVs.
test_detection.py	src/detection/rules.py	Rule scoring, classification boundaries, threshold sensitivity.
test_ml_model.py	src/detection/ml_model.py	Model loading, encoding, probability thresholding, hybrid decisions.
test_reporting.py	src/reporting/report_writer.py	CSV output, column presence, record-<id> generation.
test_dashboard.py	src/dashboard/report_processing.py	Filter logic, KPI computation, ML column detection.

COMMAND / CODE

```
# Run full test suite
python -m pytest

# Run with verbose output
python -m pytest -v

# Run a specific test file
python -m pytest tests/test_detection.py -v
```

10 SECURITY CONSIDERATIONS

Input Validation

- All CSV input is validated against the expected UNSW-NB15 schema before processing.
- Missing required columns raise a descriptive ValueError rather than silently producing wrong results.
- No arbitrary code execution paths exist in the CSV parser.

Model Artifact Security

- The .pkl model is loaded using joblib, which deserializes Python objects. Only load .pkl files from trusted sources. Do not accept .pkl files from untrusted third parties.
- The model file is version-controlled and sha-checked during CI (recommended practice for production).

Data Privacy

- The included dataset contains no real user PII. IP addresses in raw log mode are processed in-memory and not transmitted.
- Generated CSV reports should be treated as sensitive operational data and stored with appropriate access controls.

11 LIMITATIONS & FUTURE WORK

Current Limitations

- The ML model requires all 42 UNSW-NB15 features; raw logs with only srcip/dstip/dsport cannot use ML scoring.
- The cleaned dataset does not include real source IP addresses; raw log support is a separate code path.
- The system processes data in batch mode; real-time streaming is not currently supported.
- Model retraining requires the full notebook environment (Jupyter + GPU recommended for CatBoost).

Proposed Future Enhancements

Enhancement	Priority	Description
Real-time Kafka stream ingestion	High	Consume live network telemetry from Kafka topics instead of batch CSV.
REST API endpoint	High	FastAPI wrapper to expose detection as a microservice.
Automated model retraining	Medium	Scheduled retraining pipeline when dataset drift is detected.
PCAP file support	Medium	Parse raw .pcap captures directly using scapy/dpkt.
Alert export to SIEM	Medium	Push high-risk alerts to Splunk/Elastic via syslog or API.
Explainability (SHAP)	Low	Add SHAP value explanations to ML prediction outputs.
Multi-tenant dashboard	Low	Role-based access control and multi-user report management.

12 REFERENCES & DEPENDENCIES

Dataset Reference

- N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," MilCIS, Canberra, Australia, 2015.
- Australian Centre for Cyber Security (ACCS), University of New South Wales Canberra.

Software Libraries

Library	License	Reference
scikit-learn	BSD-3	Pedregosa et al., JMLR 12, 2011.
XGBoost	Apache-2	Chen & Guestrin, KDD 2016.
LightGBM	MIT	Ke et al., NeurIPS 2017.
CatBoost	Apache-2	Prokhorenkova et al., NeurIPS 2018.
imbalanced-learn	MIT	Lemaitre et al., JMLR 18(17), 2017.
pandas	BSD-3	McKinney, SciPy Proceedings 2010.
Streamlit	Apache-2	streamlit.io
joblib	BSD-3	Joblib Development Team, 2020.

Project Repository

- GitHub: <https://github.com/rajrathaur4905> (Team Lead — Raj Rathaur)
- Hackathon: Dell Technologies Innovation Hackathon 2024